Manual

2026 年 7 月 2 日

1 Python 速通

目标

能看懂别人的 Python 代码.

用法

按顺序读 -> 运行代码 -> 看输 ?-> 读注 ?

1.1 变量与数据类型

变量

变量 ?' 贴了标签的盒 ?'. 将数据放入盒 ? 贴上标签, 之后通过标签即可取出数据.

基本数据类型

- `int`: 整数, ?`5`, `-3`
- `float`: 小数, ?`3.14`, `-0.5`
- `str`: 字符 ? 用引号包起来
- `bool`: 布尔 ? ?`True` ?`False`

[71]:
```python
# 变量
age = 20
print(age, type(age))

price = 9.99
print(price, type(price))

name = "小明"
print(name, type(name))
```

```
is_student = True
print(is_student, type(is_student))
```

```
20 <class 'int'>
9.99 <class 'float'>
小明 <class 'str'>
True <class 'bool'>
```

[72]:
```
# 变量重赋值
x = 10
print("x =", x)
x = 20
print("x =", x)

a = 5
b = a
print("a =", a, "b =", b)

# 变量命名：字母/数字/下划 ？数字不能开头
my_name = "小红"
age2 = 25
print(my_name, age2)
```

```
x = 10
x = 20
a = 5 b = 5
小红 25
```

1.2 基础运算

Python 可直接做数学运算.

算术运算符

`+ - * /` – 加减乘除

`//` – 整除, `%` – 取余, `**` – 乘方

比较运算符 (结果恒为 `True` ?`False`)

`== != > < >= <=`

逻辑运算符

and: 两边都真才真; or: 一边真即真; not: 取反

[73]:
```
# 算术运算
print("3 + 2 =", 3 + 2)
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("2 ** 3 =", 2 ** 3)

# 优先级
print("2 + 3 * 4 =", 2 + 3 * 4)
print("(2 + 3) * 4 =", (2 + 3) * 4)
```

```
3 + 2 = 5
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
2 ** 3 = 8
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
```

[74]:
```
# 比较运算
print("5 == 3:", 5 == 3)
print("5 != 3:", 5 != 3)
print("5 > 3:", 5 > 3)
print("5 <= 4:", 5 <= 4)

# 逻辑运算
age = 20
has_id = True
print("age >= 18 and has_id:", age >= 18 and has_id)

is_weekend = False
is_holiday = True
print("is_weekend or is_holiday:", is_weekend or is_holiday)

is_rainy = True
print("not is_rainy:", not is_rainy)
```

```
5 == 3: False
5 != 3: True
5 > 3: True
5 <= 4: False
age >= 18 and has_id: True
is_weekend or is_holiday: True
not is_rainy: False
```

1.3 字符串操作

字符串

一串文 ? 用引号包起来.

常见操作

- + 拼接, * 重复
- len(s) 取长 ?
- s[i] 取第 i 个字 ?(下标 ?0 开 ?
- s[i:j] 切片 (?i 不含 j)
- s.upper(), s.lower() 大小写转 ?
- s.strip() 去掉两端空格
- s.split(c) ?c 分割为列 ?
- c.join(list) ?c 合并列表为字符串

[75]:
```python
# 字符串拼接与重复
s1 = "你好"
s2 = "世界"
print("s1 + s2:", s1 + s2)
print("'哈' * 3:", "哈" * 3)

# len()
text = "Hello Python"
print("len(text):", len(text))

# 下标与切片
word = "Python"
print("word[0]:", word[0])
print("word[-1]:", word[-1])
print("word[0:3]:", word[0:3])
```

```
print("word[2:]:", word[2:])
print("word[-3:]:", word[-3:])
```

```
s1 + s2: 你好世界
'? * 3: 哈哈 ?
len(text): 12
word[0]: P
word[-1]: n
word[0:3]: Pyt
word[2:]: thon
word[-3:]: hon
```

[76]:
```
# 大小写转换
msg = "  Hello, Python!  "
print("upper:", msg.upper())
print("lower:", msg.lower())
print("strip:", msg.strip())

# 替换与查找
s = "I love Python"
print("replace:", s.replace("love", "like"))
print("find:", s.find("Python"))
print("startswith:", s.startswith("I"))

# 分割与合并
csv_line = "apple,banana,orange"
fruits = csv_line.split(",")
print("split:", fruits)

joined = " + ".join(fruits)
print("join:", joined)
```

```
upper:   HELLO, PYTHON!
lower:   hello, python!
strip: Hello, Python!
replace: I like Python
find: 7
startswith: True
split: ['apple', 'banana', 'orange']
join: apple + banana + orange
```

1.4 列表, 元组与字典

三种容器对比

- 列表 list: [1, 2, 3], 有序可改
- 元组 tuple: (1, 2, 3), 有序不可 ？
- 字典 dict: {"key": "value"}, 键值对

[77]:
```
# 列表
fruits = ["苹果", "香蕉", "橘子"]
numbers = [1, 2, 3, 4, 5]
mixed = ["文字", 123, True, 3.14]
print("fruits:", fruits)

print("fruits[0]:", fruits[0])
print("fruits[-1]:", fruits[-1])
print("fruits[:2]:", fruits[:2])

fruits[1] = "西瓜"
fruits.append("葡萄")
fruits.insert(1, "草莓")
print("after modify:", fruits)

fruits.remove("苹果")
last = fruits.pop()
print("after pop:", last, fruits)

print("len:", len(fruits))
```

```
fruits: ['苹果', '香蕉', '橘子']
fruits[0]: 苹果
fruits[-1]: 橘子
fruits[:2]: ['苹果', '香蕉']
after modify: ['苹果', '草莓', '西瓜', '橘子', '葡萄']
after pop: 葡萄 ['草莓', '西瓜', '橘子']
len: 3
```

[78]:
```
# 元组
days = ("周一", "周二", "周三", "周四", "周五", "周六", "周日")
print("days:", days)
print("days[2]:", days[2])
```

```
print("len:", len(days))

# 拆包
def get_point():
    return (3, 4)

x, y = get_point()
print("x =", x, "y =", y)
```

```
days: ('周一', '周二', '周三', '周四', '周五', '周六', '周日')
days[2]: 周三
len: 7
x = 3 y = 4
```

[79]:
```
# 字典
student = {"name": "小明", "age": 20, "score": 95.5}
print("student:", student)
print("student['name']:", student["name"])

student["gender"] = "男"
student["score"] = 98
del student["age"]
print("after modify:", student)

print("'name' in student:", "name" in student)

print("keys:", list(student.keys()))
print("values:", list(student.values()))

print("get:", student.get("score", "没找到"))
print("get missing:", student.get("xyz", "不存在"))
```

```
student: {'name': '小明', 'age': 20, 'score': 95.5}
student['name']: 小明
after modify: {'name': '小明', 'score': 98, 'gender': '?}
'name' in student: True
keys: ['name', 'score', 'gender']
values: ['小明', 98, '?]
get: 98
get missing: 不存 ?
```

1.5 条件判断与循环

条件判断

若下雨则带伞, 否则不带. 程序通过条件决定执行路径.

循环

?50 个同学依次算成绩, 不必 ?50 遍代 ?

[80]:
```python
# if / elif / else
score = 85
if score >= 90:
    print("优秀")
elif score >= 80:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")

# and / or
age = 25
has_ticket = True
if age >= 18 and has_ticket:
    print("请进")

# 三元表达式
x = 10
result = "正数" if x > 0 else "非正数"
print(result)
```

```
良好
请进
正数
```

[81]:
```python
# for 循环
fruits = ["苹果", "香蕉", "橘子", "西瓜"]
for fruit in fruits:
    print(fruit)

# range
```

```
for i in range(5):
    print(i, end=" ")
print()

# enumerate
names = ["小明", "小红", "小刚"]
for index, name in enumerate(names):
    print(index, name)

# 遍历字典
student = {"name": "小明", "age": 20, "score": 95}
for key, value in student.items():
    print(key, value)
```

```
苹果
香蕉
橘子
西瓜
0 1 2 3 4
0 小明
1 小红
2 小刚
name 小明
age 20
score 95
```

[82]:
```
# while
count = 5
while count > 0:
    print("倒数:", count)
    count -= 1

# break
numbers = [1, 5, 3, 8, 2]
for n in numbers:
    if n == 3:
        break
    print(n)
```

```python
# continue
for i in range(10):
    if i % 2 != 0:
        continue
    print(i)

# 列表推导式
squares = [x**2 for x in range(5)]
print("squares:", squares)

even_squares = [x**2 for x in range(10) if x % 2 == 0]
print("even_squares:", even_squares)
```

```
倒数: 5
倒数: 4
倒数: 3
倒数: 2
倒数: 1
1
5
0
2
4
6
8
squares: [0, 1, 4, 9, 16]
even_squares: [0, 4, 16, 36, 64]
```

1.6 函数

函数

将一段代码打 ？起名, 之后通过名字反复调用.

避免重复写代 ？方便修改, 提高可读 ？

[83]:
```python
# 定义函数
def say_hello():
    print("你好!")
```

```
say_hello()

def greet(name):
    print(f"{name}, 你好!")

greet("小明")
greet("小红")

def add(a, b):
    return a + b

print("3 + 5 =", add(3, 5))
print("10 + 20 =", add(10, 20))
```

```
你好!
小明, 你好!
小红, 你好!
3 + 5 = 8
10 + 20 = 30
```

[84]:
```
# 默认参数
def repeat_msg(msg, times=3):
    for i in range(times):
        print(msg)

repeat_msg("加油!")
repeat_msg("加油!", 2)

# 多返回值
def min_max(numbers):
    return min(numbers), max(numbers)

nums = [3, 7, 1, 9, 4]
min_val, max_val = min_max(nums)
print("min:", min_val, "max:", max_val)

# 全局 vs 局部
x = 10
def my_func():
```

```
    x = 5
    print("内部:", x)

my_func()
print("外部:", x)

def add_one():
    global count
    count = count + 1

count = 0
add_one()
add_one()
print("count:", count)
```

```
加油!
加油!
加油!
加油!
加油!
min: 1 max: 9
内部: 5
外部: 10
count: 2
```

1.7 模块导入

Python 自带许多工具 ?(标准 ?, ?import 即可使用.

三种导入方式

- import math – 导入整个模块, ?math.sqrt(9)
- from math import sqrt – 只导入所需函数
- import math as m – 起别 ?

[85]:
```
import math
import random
from datetime import datetime, timedelta

print("pi:", math.pi)
```

```
print("sqrt(16):", math.sqrt(16))

print("random int:", random.randint(1, 10))
print("random choice:", random.choice(["苹果", "香蕉", "橘子"]))

now = datetime.now()
print("now:", now)
print("7 天后:", now + timedelta(days=7))
```

```
pi: 3.141592653589793
sqrt(16): 4.0
random int: 10
random choice: 苹果
now: 2026-07-02 07:44:01.514618
7 天后: 2026-07-09 07:44:01.514618
```

1.8 常用内置函数

Python 内置许多实用函数, 无需 import 即可直接使用.

[86]:
```
# type / isinstance
print(type(123), isinstance(123, int))
print(type("abc"), isinstance("abc", str))

# len
print(len("Python"), len([1, 2, 3, 4]))

# range
print(list(range(5)))
print(list(range(2, 6)))
print(list(range(0, 10, 2)))

# enumerate
for i, f in enumerate(["苹果", "香蕉", "橘子"]):
    print(i, f)

# zip
for name, score in zip(["小明", "小红"], [85, 92]):
    print(name, score)
```

```python
# sorted
nums = [3, 1, 4, 1, 5, 9]
print(sorted(nums))
print(sorted(nums, reverse=True))

# map / filter
numbers = [1, 2, 3, 4, 5]
print(list(map(lambda x: x*2, numbers)))
print(list(filter(lambda x: x%2==0, numbers)))

# any / all
checks = [True, False, True]
print(any(checks), all(checks))
```

```
<class 'int'> True
<class 'str'> True
6 4
[0, 1, 2, 3, 4]
[2, 3, 4, 5]
[0, 2, 4, 6, 8]
0 苹果
1 香蕉
2 橘子
小明 85
小红 92
[1, 1, 3, 4, 5, 9]
[9, 5, 4, 3, 1, 1]
[2, 4, 6, 8, 10]
[2, 4]
True False
```

1.9 文件读写

?`with open()` 打开文件, 自动关闭.

- `'r'` 读取, `'w'` 写入, `'a'` 追加

[87]:
```python
# 写入
with open("test.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n")
    f.write("第二行\n")

# 读取
with open("test.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# 逐行
with open("test.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

import os
os.remove("test.txt")
```

第一 ?
第二 ?

第一 ?
第二 ?

1.10 错误处理

常见错误类型

- NameError: 变量未定 ?
- TypeError: 类型不匹 ?
- IndexError: 下标越界
- KeyError: 字典键不存在
- ValueError: 值不合法
- FileNotFoundError: 文件未找 ?
- ZeroDivisionError: 除以 0

[88]:
```python
# NameError: 变量未定义
if 'undefined_variable' in locals():
    print(undefined_variable)
else:
```

```
    print("变量未定义")

# TypeError: 类型不匹配
print("年龄:" + str(18))

# IndexError: 下标越界
fruits = ["苹果", "香蕉", "橘子"]
index = 5
if index < len(fruits):
    print(fruits[index])
else:
    print(f"下标{index}越界")
```

```
变量未定 ?
年龄:18
下标 5 越界
```

[89]:
```
# try / except
try:
    result = 10 / 0
except ZeroDivisionError:
    print("不能除以 0")

try:
    num = int("abc")
except ValueError:
    print("'abc' 不是数字")

# finally
try:
    f = open("不存在的文件.txt", "r")
except FileNotFoundError:
    print("文件不存在")
finally:
    print("清理完成")

# 安全转换
def safe_int(value, default=0):
    try:
```

```
        return int(value)
    except (ValueError, TypeError):
        return default

print(safe_int("123"))
print(safe_int("abc"))
```

```
不能除以 0
'abc' 不是数字
文件不存 ?
清理完成
123
0
```

1.11 类与面向对象

?(class) = 设计图纸; 对象 (object) = 实际产物

同一个类可产生多个不同对 ? 将数据与操作数据的方法打包在一 ?

魔法方法

- `__init__`: 构造器, 创建对象时自动调 ?
- `__str__`: `print()` 时调 ?
- `__eq__`: `==` 比较时调 ?
- `__lt__`: `<` 比较时调 ?(用于排序)

[90]:
```
class Student:
    def __init__(self, name, age, score):
        self.name = name
        self.age = age
        self.score = score

    def introduce(self):
        return f"我叫{self.name}, {self.age}? {self.score}分"

    def is_pass(self):
        return self.score >= 60

s1 = Student("小明", 20, 95)
s2 = Student("小红", 21, 58)
```

```
print(s1.introduce())
print(s1.is_pass())
print(s2.is_pass())
```

```
我叫小明，20？ 95？
True
False
```

[91]:
```
class Animal:
    def __init__(self, name):
        self.name = name
    def make_sound(self):
        return "..."

class Dog(Animal):
    def make_sound(self):
        return "汪汪!"

class Cat(Animal):
    def make_sound(self):
        return "喵喵 ~"

dog = Dog("旺财")
cat = Cat("咪咪")
print(dog.make_sound())
print(cat.make_sound())
```

```
汪汪!
喵喵 ~
```

[92]:
```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price
    def __str__(self):
        return f"《{self.title}》"
    def __eq__(self, other):
        return self.title == other.title
```

```
    def __lt__(self, other):
        return self.price < other.price

b1 = Book("Python 编程", "张三", 59)
b2 = Book("Python 编程", "张三", 59)
b3 = Book("数据结构", "李四", 45)

print(b1)
print(b1 == b2)
print(b1 == b3)
print(sorted([b1, b3])[0])
```

```
《Python 编程 ？
True
False
《数据结构 ？
```

1.12 lambda 表达式与高阶函数

lambda = 无名一次性函数

lambda 参数: 表达式

常用 ?sorted/map/filter/reduce 等函数中作为参数.

[93]:
```
# lambda
add = lambda a, b: a + b
print(add(3, 5))

# sort ?key
students = [
    {"name": "小明", "score": 95},
    {"name": "小红", "score": 58},
    {"name": "小刚", "score": 78},
]

by_score = sorted(students, key=lambda s: s["score"])
print(by_score)

by_score_desc = sorted(students, key=lambda s: s["score"], reverse=True)
```

```
print(by_score_desc)
```

```
8
[{'name': '小红', 'score': 58}, {'name': '小刚', 'score': 78}, {'name': '小明',
'score': 95}]
[{'name': '小明', 'score': 95}, {'name': '小刚', 'score': 78}, {'name': '小红',
'score': 58}]
```

[94]:
```
from functools import reduce

nums = [1, 2, 3, 4, 5]

# map / filter / reduce
print(list(map(lambda x: x*2, nums)))
print(list(filter(lambda x: x%2==0, nums)))
print(reduce(lambda a, b: a+b, nums))
print(reduce(lambda a, b: a*b, nums))

# 一 ? 偶数平方后排序
result = sorted(map(lambda x: x**2, filter(lambda x: x%2==0, nums)))
print(result)
```

```
[2, 4, 6, 8, 10]
[2, 4]
15
120
[4, 16]
```

1.13 正则表达 ?(re)

正则 = 模式匹配语言

用于查找/提取/替换符合某种模式的字符串.

常用符号

- . 任意字符, * 零次或多 ? + 一次或多次, ? 零次或一 ?
- \d 数字, \w 字母/数字/下划 ? \s 空白
- ^ 开 ? $ 结尾, [abc] a/b/c 之一, (x|y) x ?y

[95]:
```python
import re

# findall
text = "电话: 138-1234-5678, 139-8765-4321"
print(re.findall(r'\d{3}-\d{4}-\d{4}', text))

# search + 分组
text2 = "日期: 2024-03-15"
m = re.search(r'(\d{4})-(\d{2})-(\d{2})', text2)
print(m.group(0), m.group(1), m.group(2))

# sub
text3 = "密码: pass123"
print(re.sub(r'密码:\w+', '密码:***', text3))
```

```
['138-1234-5678', '139-8765-4321']
2024-03-15 2024 03
密码: pass123
```

[96]:
```python
text = """
张三: zhangsan@company.com
李四: lisi@test.org.cn
"""
emails = re.findall(r'[\w.]+@[\w.]+\.\w+', text)
print(emails)

# 验证手机号
def validate_phone(p):
    return bool(re.match(r'^1[3-9]\d{9}$', p))

print(validate_phone("13812345678"))
print(validate_phone("1234"))

# 提取 HTML 链接
html = '<a href="https://www.baidu.com"> 百度 </a>'
links = re.findall(r'href=[\'"]?(https?://[^\'" >]+)', html)
print(links)
```

```
['zhangsan@company.com', 'lisi@test.org.cn']
True
```

```
False
['https://www.baidu.com']
```

1.14 装饰 ?(Decorator)

装饰 ?= 给函数加一层包装

不修改原函数代码, 为其添加额外功能.

写法: @ 装饰器名置于函数定义上方.

[97]:
```python
# 装饰器
def my_decorator(func):
    def wrapper():
        print("调用前")
        func()
        print("调用后")
    return wrapper

@my_decorator
def say_hello():
    print("你好")

say_hello()
```

```
调用 ?
你好
调用 ?
```

[98]:
```python
import time

# 计时装饰器
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__}: {time.time()-start:.4f}s")
        return result
    return wrapper

@timer
```

```
def slow_sum(n):
    return sum(range(n))

@timer
def fast_sum(n):
    return n*(n-1)//2

print(slow_sum(1000000))
print(fast_sum(1000000))
```

```
slow_sum: 0.0060s
499999500000
fast_sum: 0.0000s
499999500000
```

1.15 生成器与迭代器

生成 ?(Generator)

?`yield` 而非 `return`, 一次只生成一个 ? 不一次性全部算 ?

节省内存, 适合处理超大数据.

生成器表达式

?() 代替 []: `(x**2 for x in range(10))`

[99]:
```
# 生成器
def my_range(n):
    i = 0
    while i < n:
        yield i
        i += 1

for x in my_range(5):
    print(x, end=" ")
print()

# 生成器只能用一次
gen = my_range(3)
print(list(gen))
```

```
print(list(gen))  # 空
```

```
0 1 2 3 4
[0, 1, 2]
[]
```

[100]:
```
# 斐波那契（无限生成器）
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
for i in range(10):
    print(next(fib), end=" ")
print()

# 生成器表达式
gen = (x**2 for x in range(5))
print(list(gen))

# 分页
def paginate(data, n):
    for i in range(0, len(data), n):
        yield data[i:i+n]

students = [f"学生{i}" for i in range(1, 13)]
for page in paginate(students, 5):
    print(page)
```

```
0 1 1 2 3 5 8 13 21 34
[0, 1, 4, 9, 16]
['学生 1', '学生 2', '学生 3', '学生 4', '学生 5']
['学生 6', '学生 7', '学生 8', '学生 9', '学生 10']
['学生 11', '学生 12']
```

1.16 第三方库

三大常用库

- numpy: 数组/矩阵运算
- pandas: 表格数据处理
- requests: 网络请求 (HTTP)

[101]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr, arr.shape)

print(arr + 10)
print(arr * 2)
print(np.sqrt(arr))

matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix.shape)
print(matrix[0])
print(matrix[:, 1])

print(np.zeros((2, 3)))
print(np.eye(3))

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(data.sum(), data.mean(), data.max())
```

```
[1 2 3 4 5] (5,)
[11 12 13 14 15]
[ 2  4  6  8 10]
[1.         1.41421356 1.73205081 2.         2.23606798]
(2, 3)
[1 2 3]
[2 5]
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
55 5.5 10
```

[102]:
```
import pandas as pd
```

```
data = {
    "姓名": ["小明", "小红", "小刚", "小李"],
    "年龄": [20, 21, 19, 22],
    "成绩": [95, 58, 78, 88],
}
df = pd.DataFrame(data)
print(df)

print(df[df["成绩"] >= 60])
print(df.sort_values("成绩", ascending=False))

print("mean:", df["成绩"].mean())
print("max:", df["成绩"].max())

df["是否及格"] = df["成绩"] >= 60
print(df)
```

```
   姓名  年龄  成绩
0  小明  20  95
1  小红  21  58
2  小刚  19  78
3  小李  22  88
   姓名  年龄  成绩
0  小明  20  95
2  小刚  19  78
3  小李  22  88
   姓名  年龄  成绩
0  小明  20  95
3  小李  22  88
2  小刚  19  78
1  小红  21  58
mean: 79.75
max: 95
   姓名  年龄  成绩    是否及格
0  小明  20  95   True
1  小红  21  58  False
2  小刚  19  78   True
3  小李  22  88   True
```

[103]:
```python
import requests

# GET
resp = requests.get("https://api.github.com", timeout=5)
print(resp.status_code)
print(list(resp.json().keys())[:5])

# POST
resp = requests.post("https://httpbin.org/post",
    json={"user": "test"}, timeout=5)
print(resp.json()["json"])

# 参数
resp = requests.get("https://httpbin.org/get",
    params={"q": "python", "page": 1}, timeout=5)
print(resp.url)
```

```
200
['current_user_url', 'current_user_authorizations_html_url',
'authorizations_url', 'code_search_url', 'commit_search_url']
{'user': 'test'}
https://httpbin.org/get?q=python&page=1
```

1.17 多线程与异步编程

三种并发方式

- 多线 ?threading: 适合 I/O 密集 ?(下载/读写)
- 线程安全: ?Lock 避免数据竞争
- 异步 async/await: 单线程高并发, 适合网络请求

[104]:
```python
import threading, time

def work(name, t):
    print(f"{name} 开始")
    time.sleep(t)
    print(f"{name} 完成")

# 单线程
start = time.time()
```

```
work("A", 1)
work("B", 1)
print("串行:", time.time()-start)

# 多线程
start = time.time()
t1 = threading.Thread(target=work, args=("A", 1))
t2 = threading.Thread(target=work, args=("B", 1))
t1.start(); t2.start()
t1.join(); t2.join()
print("并行:", time.time()-start)
```

```
A 开 ?
A 完成
B 开 ?
B 完成
串行: 2.001854658126831
A 开 ?
B 开 ?
B 完成 A 完成

并行: 1.0028059482574463
```

[105]:
```
from threading import Thread, Lock

# 无锁: 数据竞争
counter = 0
def bad():
    global counter
    for _ in range(100000):
        counter += 1

ts = [Thread(target=bad) for _ in range(2)]
[t.start() for t in ts]
[t.join() for t in ts]
print("无锁:", counter)

# 有锁: 安全
counter = 0
```

```
lock = Lock()
def safe():
    global counter
    for _ in range(100000):
        with lock:
            counter += 1

ts = [Thread(target=safe) for _ in range(2)]
[t.start() for t in ts]
[t.join() for t in ts]
print("有锁:", counter)
```

无锁: 200000
有锁: 200000

[106]:
```
import asyncio, time

async def task(name, t):
    print(f"{name} 开始")
    await asyncio.sleep(t)
    print(f"{name} 完成")

# 并发
start = time.time()
await asyncio.gather(
    task("A", 2), task("B", 2), task("C", 2),
)
print("总耗时:", time.time()-start)
```

A 开 ?
B 开 ?
C 开 ?
A 完成
B 完成
C 完成
总耗时: 2.0128684043884277